Shiv Nadar University Chennai

Degree & Branch	B.Tech Artificial Intelligence & Data Science	Semester V
Subject Code & Name	CS3807 – Deep Learning Laboratory	AY: 2026–27

Experiment 1

Implementation of a Single Layer Perceptron for Binary Classification

1. Objective

The objective of this experiment is twofold. First, to understand the working principle of an Artificial Neuron and implement a Single Layer Perceptron from scratch for binary classification, using the Banknote Authentication dataset to train and evaluate the model, studying the effect of feature normalization, analysing the learning behaviour using different plots, and evaluating the classifier using standard performance metrics such as Accuracy, Precision, Recall and F1-score. Second, to implement the Perceptron Learning Algorithm from scratch in Python and use it to learn the OR, AND and NOT logic gates, recording the weights and bias after every individual weight update, visualizing the decision boundary at every stage of learning, and verifying that a Single Layer Perceptron can successfully converge on these gates because they represent linearly separable functions.

2. Background Theory

Artificial Neural Networks are computational models inspired by the functioning of biological neurons. The fundamental processing unit of an Artificial Neural Network is the Artificial Neuron. Each neuron receives several input features, multiplies them with trainable weights, adds a bias value and passes the result through an activation function to generate an output.

The Single Layer Perceptron proposed by Frank Rosenblatt is one of the earliest supervised learning algorithms. It is capable of solving only linearly separable binary classification problems. During training, the perceptron iteratively updates its weights and bias using the perceptron learning rule until the classification error is minimized.

The Perceptron, introduced by Frank Rosenblatt, is one of the earliest supervised learning algorithms capable of solving binary classification problems that are linearly separable. Logic gates such as OR, AND and NOT are classical examples of linearly separable Boolean functions, since a single straight line (in two input dimensions) or a single threshold point (in one input dimension) is sufficient to correctly separate the output classes 0 and 1.

During training, the perceptron computes a weighted sum of its inputs, passes this sum through a Step Activation Function, and compares the resulting prediction with the true target value. Whenever the prediction is incorrect, the weights and bias are updated immediately (online learning), and training continues until an entire pass over all input combinations produces zero misclassifications, at which point the perceptron is said to have converged.

Mathematical Formulation

The weighted sum of the inputs is

$$z = \mathbf{w}^T \mathbf{x} + b$$

where

- $\mathbf{x}$ denotes the input feature vector.
- $\mathbf{w}$ denotes the weight vector.
- b denotes the bias.
- z denotes the weighted sum.

The output of the perceptron is

$$\hat{y} = f(z)$$

where $f(z)$ represents the activation function.
The perceptron learning rule updates the weights according to

$$\mathbf{w}_{new} = \mathbf{w}_{old} + \eta(y - \hat{y})\mathbf{x}$$

The bias is updated using

$$b_{new} = b_{old} + \eta(y - \hat{y})$$

where

- η is the learning rate.
- y is the actual class.
- $\hat{y}$ is the predicted class.

The weighted sum of the inputs is

$$z = \mathbf{w}^T \mathbf{x} + b$$

where $\mathbf{x}$ is the input feature vector, $\mathbf{w}$ is the weight vector and b is the bias. The predicted output is

$$\hat{y} = f(z)$$

where $f(z)$ is the Step Activation Function. Whenever $\hat{y} \neq y$, the weights and bias are updated according to the Perceptron Learning Rule

$$\mathbf{w}_{new} = \mathbf{w}_{old} + \eta(y - \hat{y})\mathbf{x}$$

$$b_{new} = b_{old} + \eta(y - \hat{y})$$

where η is the learning rate, y is the true class and $\hat{y}$ is the predicted class. In this experiment, $\eta = 0.1$ and the weights and bias were both initialized to zero.

3. Activation Function

An activation function determines the output of a neuron based on its weighted input. In this experiment, the Step Activation Function is employed because the problem involves binary classification. The function produces either 0 or 1 depending on the value of the weighted sum.

$$f(z) = \begin{cases} 1, & z \geq 0 \\ 0, & z < 0 \end{cases}$$

The Step Activation Function is simple and computationally efficient. However, it is not differentiable and therefore cannot be used with gradient-based optimization algorithms such as backpropagation.

Activation Function	**Output Range**	**Typical Application**
Step	$\{0, 1\}$	Classical Perceptron
Sigmoid	$(0, 1)$	Binary Classification Output Layer
Tanh	$(-1, 1)$	Hidden Layers
ReLU	$[0, \infty)$	Deep Neural Networks
Leaky ReLU	$(-\infty, \infty)$	Avoiding Dying ReLU
Softmax	$(0, 1)$	Multi-class Classification

Although several activation functions exist, only the Step Activation Function is used in this experiment because the Single Layer Perceptron performs binary classification on linearly separable data.

The Step Activation Function was used, since all three gates involve binary classification.

$$f(z) = \begin{cases} 1, & z \geq 0 \\ 0, & z < 0 \end{cases}$$

This function is simple, computationally inexpensive, and matches the classical formulation of the Rosenblatt Perceptron.

4. Dataset Description

The experiment uses the **Banknote Authentication Dataset** obtained from the UCI Machine Learning Repository. The dataset consists of statistical features extracted from digital images of genuine and forged banknotes using wavelet transforms.

Property	**Value**
Dataset Name	Banknote Authentication Dataset
Source	UCI Machine Learning Repository
Total Samples	1372
Input Features	4
Target Classes	2
Missing Values	None
Learning Type	Supervised Learning
Problem Type	Binary Classification

Feature Description

Feature	**Description**
Variance	Variance of the wavelet transformed image.
Skewness	Degree of asymmetry in the image distribution.
Curtosis	Sharpness or flatness of the probability distribution.
Entropy	Measure of randomness present in the image.

Target Classes

- Class 0 – Authentic Banknote
- Class 1 – Forged Banknote

The dataset contains no missing values and is well suited for demonstrating the learning capability of a Single Layer Perceptron after feature normalization.

Unlike a real-world dataset, the "dataset" for this experiment consists of the complete truth tables of three fundamental logic gates. Each truth table represents a linearly separable binary classification problem.

x1	**x2**	**OR Output**
0	0	0
0	1	1
1	0	1
1	1	1

x1	**x2**	**AND Output**
0	0	0
0	1	0
1	0	0
1	1	1

x1	**NOT Output**
0	1
1	0

Both OR and AND are two-input gates whose outputs can be separated by a single straight line in the (x_1, x_2) plane, whereas NOT is a single-input gate whose output can be separated by a single threshold point on the number line.

5. Experimental Procedure

The experiment was carried out using Python in Google Colab. The implementation followed a systematic workflow beginning from dataset loading to model evaluation.

Task 1: Dataset Exploration

The dataset was first imported into the notebook using the Pandas library. The first five samples, dataset dimensions, missing values and descriptive statistics were obtained.

Output

- Total Samples : 1372
- Total Features : 4
- Target Classes : 2
- Missing Values : None

The descriptive statistics indicated that the dataset contains features with different numerical ranges. Therefore, feature normalization was performed before training the perceptron model.

Task 2: Exploratory Data Analysis

Exploratory Data Analysis (EDA) was carried out to understand the characteristics of the dataset before model training.

The following visualizations were generated.

- Feature Histograms
- Correlation Heatmap
- Scatter Plot
- Box Plot

These visualizations helped in identifying feature distributions, relationships among variables and the presence of outliers.

Task 3: Data Preprocessing

Before training, the dataset was preprocessed using the following steps.

1. Selected the four numerical features as input variables.
2. Selected the class column as the target variable.
3. Normalized all feature values using Min-Max Scaling.
4. Split the dataset into training and testing subsets.
5. Used an 80:20 train-test ratio.

The normalized dataset ensured that all features contributed equally during weight updates.

Task 4: Perceptron Implementation

A Single Layer Perceptron was implemented completely from scratch without using any machine learning libraries.

The implementation consisted of

- Weight Initialization
- Bias Initialization
- Step Activation Function
- Forward Propagation
- Perceptron Learning Rule
- Weight Update
- Bias Update

The weights were initialized with small random values and updated after every misclassified sample according to the perceptron learning rule.

Task 5: Model Training

The perceptron was trained for a maximum of 100 epochs.

During every epoch,

- Forward propagation was performed.
- Misclassified samples were identified.
- Weights were updated.
- Bias was updated.
- Training error was recorded.

Training continued until all epochs were completed while monitoring convergence through the number of misclassified samples.

Task 6: Model Evaluation

After training, the classifier was evaluated using

- Accuracy
- Precision
- Recall
- F1-Score
- Confusion Matrix

The evaluation metrics measured the effectiveness of the trained perceptron on unseen testing samples.

The experiment was carried out using Python in Google Colab.

Task 7: Truth Table Definition

The complete truth tables of the OR, AND and NOT gates were defined as NumPy arrays, to be used as the training data for three independent perceptrons.

Task 8: Perceptron Implementation and Training

A Single Layer Perceptron was implemented completely from scratch, without the use of any machine learning libraries. Weights and bias were initialized to zero, and the perceptron was trained separately on each gate using a learning rate of 0.1. Training proceeded sample-by-sample (online learning); whenever a sample was misclassified, the weights and bias were updated immediately and the updated values were printed to the console. Training continued until an entire epoch produced zero misclassifications.

Task 9: Decision Boundary Visualization

Immediately after every weight update, the corresponding decision boundary was recorded. For the two-input gates (OR, AND), the boundary is the straight line $w_1x_1 + w_2x_2 + b = 0$. For the single-input NOT gate, the boundary is the threshold point $x_1 = -b/w_1$ on the number line. All recorded boundaries, across all three gates, were combined into a single figure using Matplotlib subplots.

Task 10: Convergence Verification

After training, the final weights and bias of each gate were verified by checking that every training sample was correctly classified, confirming convergence in every case.

6. Source Code

The complete implementation was carried out in Python using Google Colab.

https://github.com/K-Mithran/Deep-Learning/blob/main/Lab1/DL_1.ipynb

The major stages of implementation included

1. Loading the dataset.
2. Performing exploratory data analysis.
3. Normalizing the dataset.
4. Splitting the dataset.
5. Implementing the perceptron from scratch.
6. Training the perceptron.
7. Evaluating the trained model.
8. Visualizing the learning behaviour.

The complete source code used in this experiment is available in the submitted Google Colab notebook and GitHub repository.

7. Experimental Results

7.1 Dataset Exploration

The Banknote Authentication dataset was successfully loaded into Google Colab. The dataset contains four numerical input features and one binary target class.

Property	**Observed Value**
Dataset Size	1372 Samples
Number of Features	4
Number of Classes	2
Missing Values	0
Training Samples	1097
Testing Samples	275

The dataset was found to be complete with no missing values. The descriptive statistics showed noticeable differences in feature ranges, making feature normalization necessary before training.

7.2 Data Preprocessing Results

After applying Min-Max Normalization, every feature value was transformed into the range [0,1].

The dataset was divided into

- Training Set : 80%
- Testing Set : 20%

This preprocessing step improved convergence during perceptron training.

7.3 Model Training

The perceptron was trained for 100 epochs.

During training, the number of misclassified samples gradually reduced while the weights and bias converged to stable values.

The final learned parameters are shown below.

Parameter	**Value**
Weight 1	-0.2301
Weight 2	-0.2384
Weight 3	-0.2603
Weight 4	0.0007
Bias	0.31
Maximum Epochs	100

7.4 Performance Metrics

The trained perceptron was evaluated on the testing dataset using standard classification metrics.

Metric	Value
Accuracy	98.18%
Precision	100.00%
Recall	96.03%
F1-Score	97.98%

The obtained results indicate that the perceptron successfully classified the majority of unseen testing samples. The perfect precision value indicates that every banknote predicted as forged was indeed forged, while the high recall confirms that almost all forged banknotes were correctly identified.

7.5 Learning Rate Comparison

The perceptron model was trained using three different learning rates to study their influence on model performance.

Learning Rate	Epochs	Accuracy
0.001	100	99.64%
0.010	100	99.64%
0.100	100	98.91%

The comparison shows that learning rates of 0.001 and 0.01 achieved the highest classification accuracy. Increasing the learning rate to 0.1 resulted in a slight reduction in performance because of larger parameter updates during training.

7.6 OR Gate – Weight Update Log

The perceptron was trained on the OR gate with a learning rate of 0.1, initial weights $[0, 0]$ and initial bias 0.0000.

Update	Epoch	Input	Target	Pred	Weights	Bias
1	1	(0,0)	0	1	[0.0, 0.0]	-0.1000
2	1	(0,1)	1	0	[0.0, 0.1]	0.0000
3	2	(0,0)	0	1	[0.0, 0.1]	-0.1000
4	2	(1,0)	1	0	[0.1, 0.1]	0.0000
5	3	(0,0)	0	1	[0.1, 0.1]	-0.1000

The OR gate converged after 4 epochs, with a total of 5 weight updates, reaching a final configuration of Weights $= [0.1, 0.1]$ and Bias $= -0.1000$.

7.7 AND Gate – Weight Update Log

The perceptron was trained on the AND gate with the same learning rate and zero initial weights and bias.

Update	Epoch	Input	Target	Pred	Weights	Bias
1	1	(0,0)	0	1	[0.0, 0.0]	-0.1000
2	1	(1,1)	1	0	[0.1, 0.1]	0.0000
3	2	(0,0)	0	1	[0.1, 0.1]	-0.1000
4	2	(0,1)	0	1	[0.1, 0.0]	-0.2000
5	2	(1,1)	1	0	[0.2, 0.1]	-0.1000
6	3	(0,1)	0	1	[0.2, 0.0]	-0.2000
7	3	(1,0)	0	1	[0.1, 0.0]	-0.3000
8	3	(1,1)	1	0	[0.2, 0.1]	-0.2000

The AND gate converged after 4 epochs, with a total of 8 weight updates, reaching a final configuration of Weights $= [0.2, 0.1]$ and Bias $= -0.2000$. The AND gate required more updates than the OR gate because the positive class (output 1) corresponds to only a single corner point $(1, 1)$ of the input space, requiring the decision boundary to be shifted further before the single positive example could be correctly isolated from the three negative examples.

7.8 NOT Gate – Weight Update Log

The perceptron was trained on the NOT gate, which has only a single input feature.

Update	Epoch	Input	Target	Pred	Weight	Bias
1	1	(1)	0	1	[-0.1]	-0.1000
2	2	(0)	1	0	[-0.1]	0.0000

The NOT gate converged after just 3 epochs, with only 2 weight updates in total, reaching a final configuration of Weight $= -0.1000$ and Bias $= 0.0000$. This gate converged fastest because it involves only a single input feature and only two possible training samples.

7.9 Final Learned Parameters

Gate	Total Updates	Epochs to Converge	Final Weights	Final Bias
OR	5	4	[0.1, 0.1]	-0.1000
AND	8	4	[0.2, 0.1]	-0.2000
NOT	2	3	[-0.1]	0.0000

8. Performance Summary

Dataset Size	1372
Training Samples	1097
Testing Samples	275
Features	4
Learning Rate	0.01
Epochs	100
Final Bias	0.31
Accuracy	98.18%
Precision	100.00%
Recall	96.03%
F1-Score	97.98%

Learning Rate	0.1
Weight/Bias Initialization	Zero
Gates Trained	OR, AND, NOT
Total Weight Updates (All Gates)	15
OR Gate Convergence	4 epochs (5 updates)
AND Gate Convergence	4 epochs (8 updates)
NOT Gate Convergence	3 epochs (2 updates)
Gates Successfully Learned	3 of 3

9. Generated Plots with Interpretation

9.1 Feature Histograms

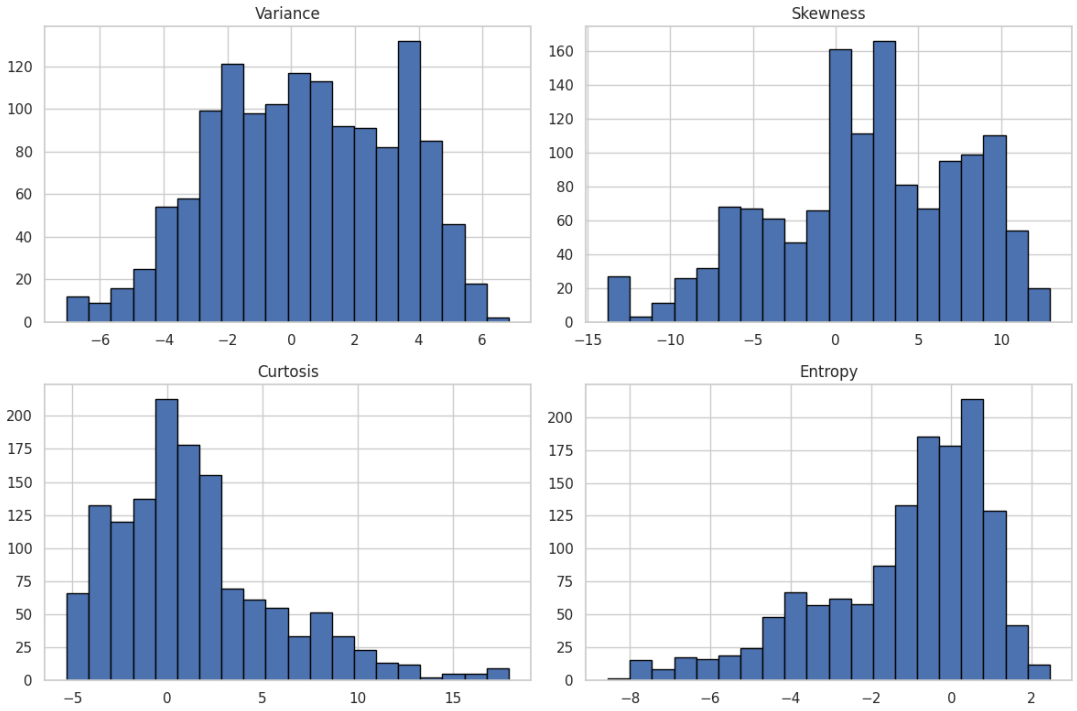


Figure 1: Feature Histograms of the Banknote Authentication Dataset

The histograms illustrate the distribution of all four numerical features present in the dataset. The features exhibit different ranges and varying degrees of skewness, indicating that normalization is necessary before training the perceptron.

9.2 Correlation Heatmap

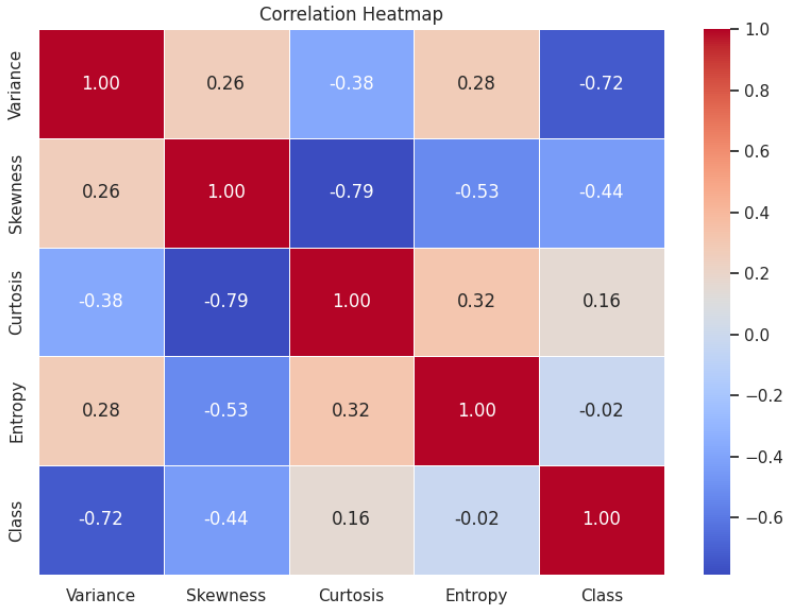


Figure 2: Correlation Heatmap

The heatmap illustrates the correlation between the input features. Highly correlated features provide useful information regarding the relationships among variables and help in understanding the overall structure of the dataset.

9.3 Scatter Plot

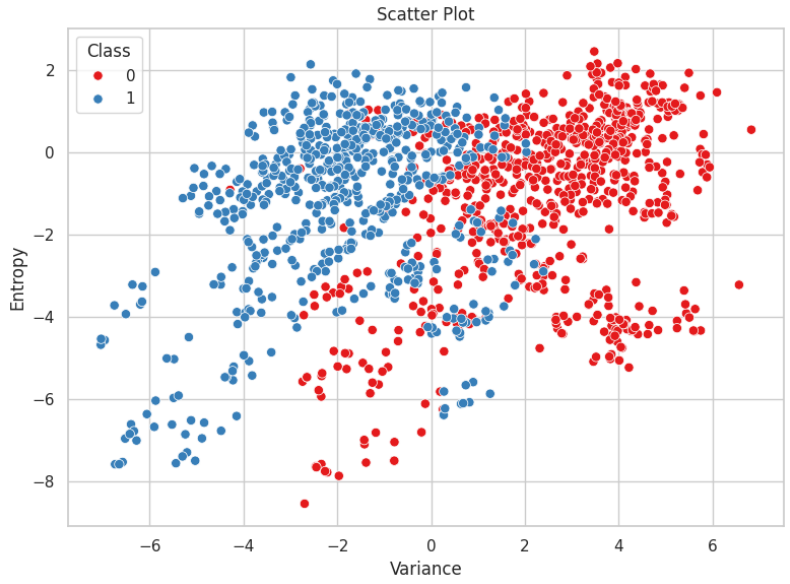


Figure 3: Scatter Plot

The scatter plot shows that the two classes are largely separable using the selected features. This justifies the use of a Single Layer Perceptron, which performs well on linearly separable datasets.

9.4 Box Plot

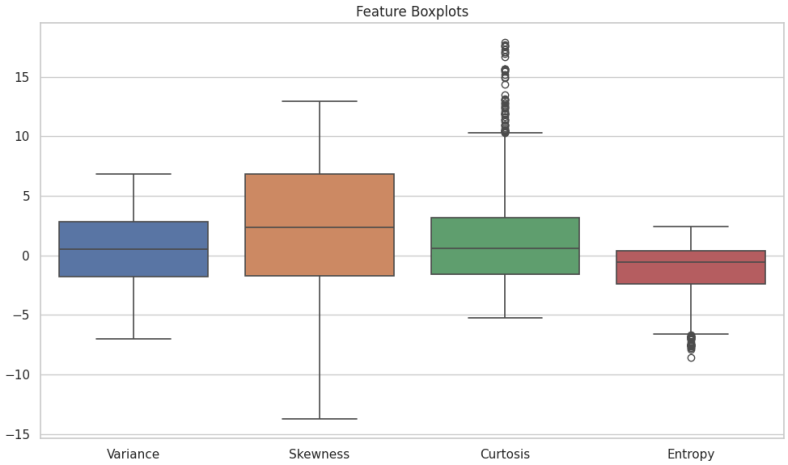


Figure 4: Box Plot

The box plot summarizes the spread of each feature and highlights the presence of a few outliers. Since Min-Max Normalization was applied before training, these outliers did not significantly affect the learning process.

9.5 Confusion Matrix

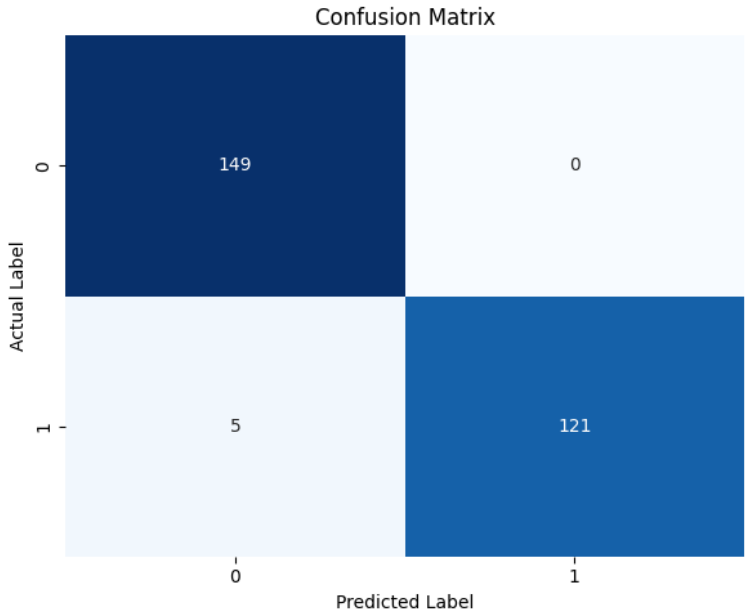


Figure 5: Confusion Matrix

The confusion matrix indicates that the perceptron correctly classified almost every testing sample. The small number of false negatives demonstrates the effectiveness of the trained model in distinguishing authentic and forged banknotes.

9.6 Training Error vs Epoch

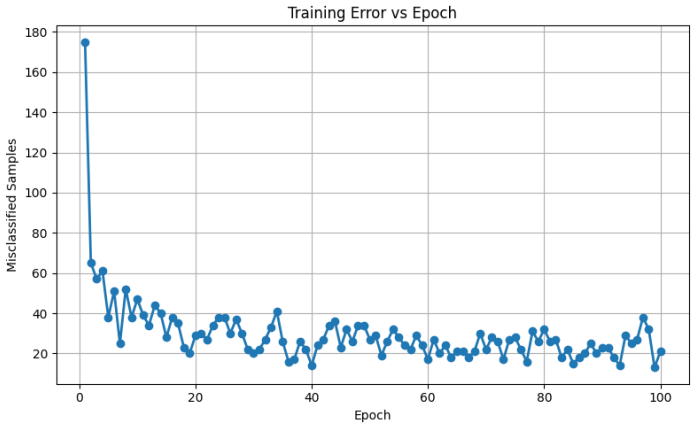


Figure 6: Training Error vs Epoch

The number of misclassified samples decreases considerably as training progresses. This behaviour indicates that the perceptron successfully learns a suitable decision boundary and gradually converges towards a stable solution.

9.7 Weight Evolution During Training

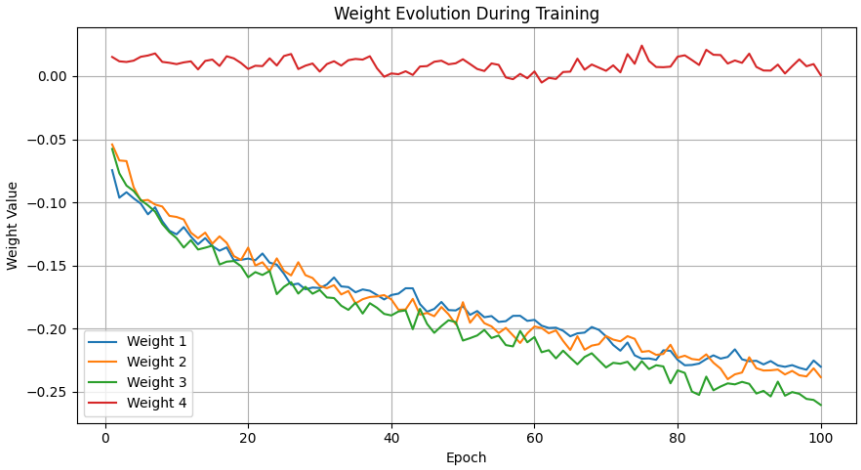


Figure 7: Weight Evolution During Training

The weight evaluation graph illustrates how the four weights are continuously adjusted during learning. Towards the final epochs the weights stabilize, indicating convergence of the learning algorithm.

9.8 Bias Evaluation

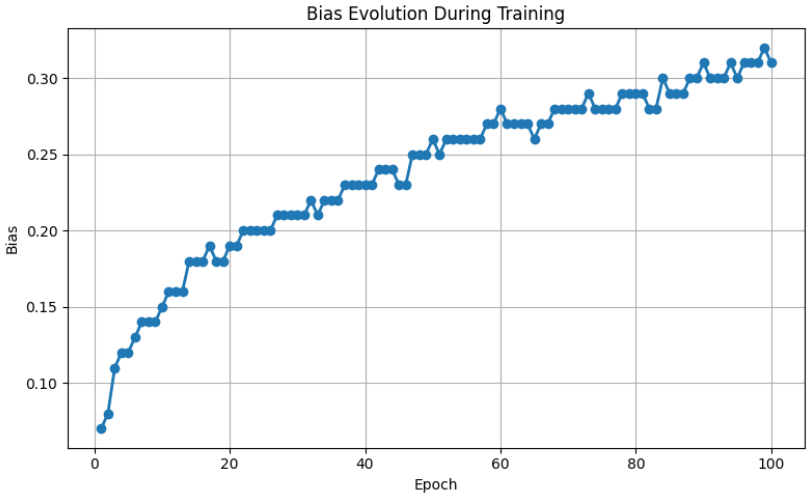


Figure 8: Bias Evaluation During Training

The bias value changes gradually during training before approaching a stable value. This demonstrates that the bias contributes significantly to shifting the decision boundary and improving classification accuracy.

9.9 Learning Rate Comparison

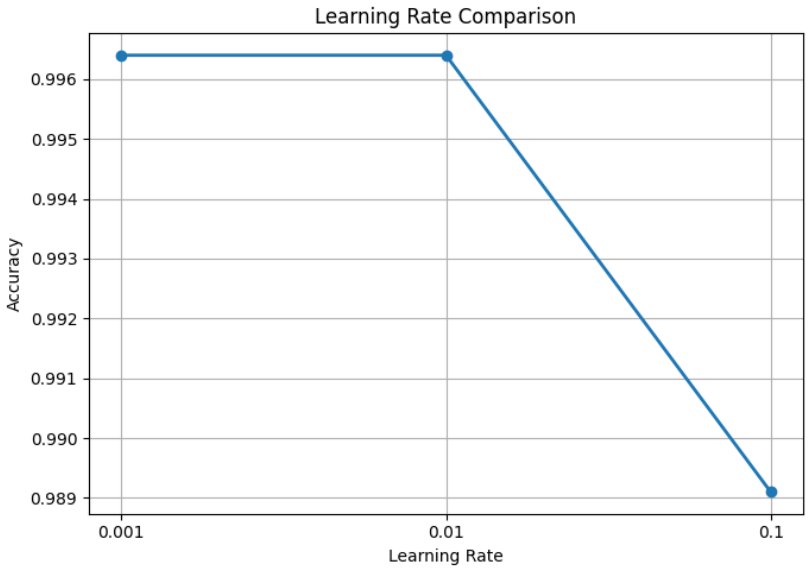


Figure 9: Learning Rate Comparison

The comparison of different learning rates indicates that both 0.001 and 0.01 achieved the highest testing accuracy of 99.64%. Increasing the learning rate to 0.1 resulted in a slight reduction in performance due to larger parameter updates during training.

9.10 Combined Decision Boundary Visualization

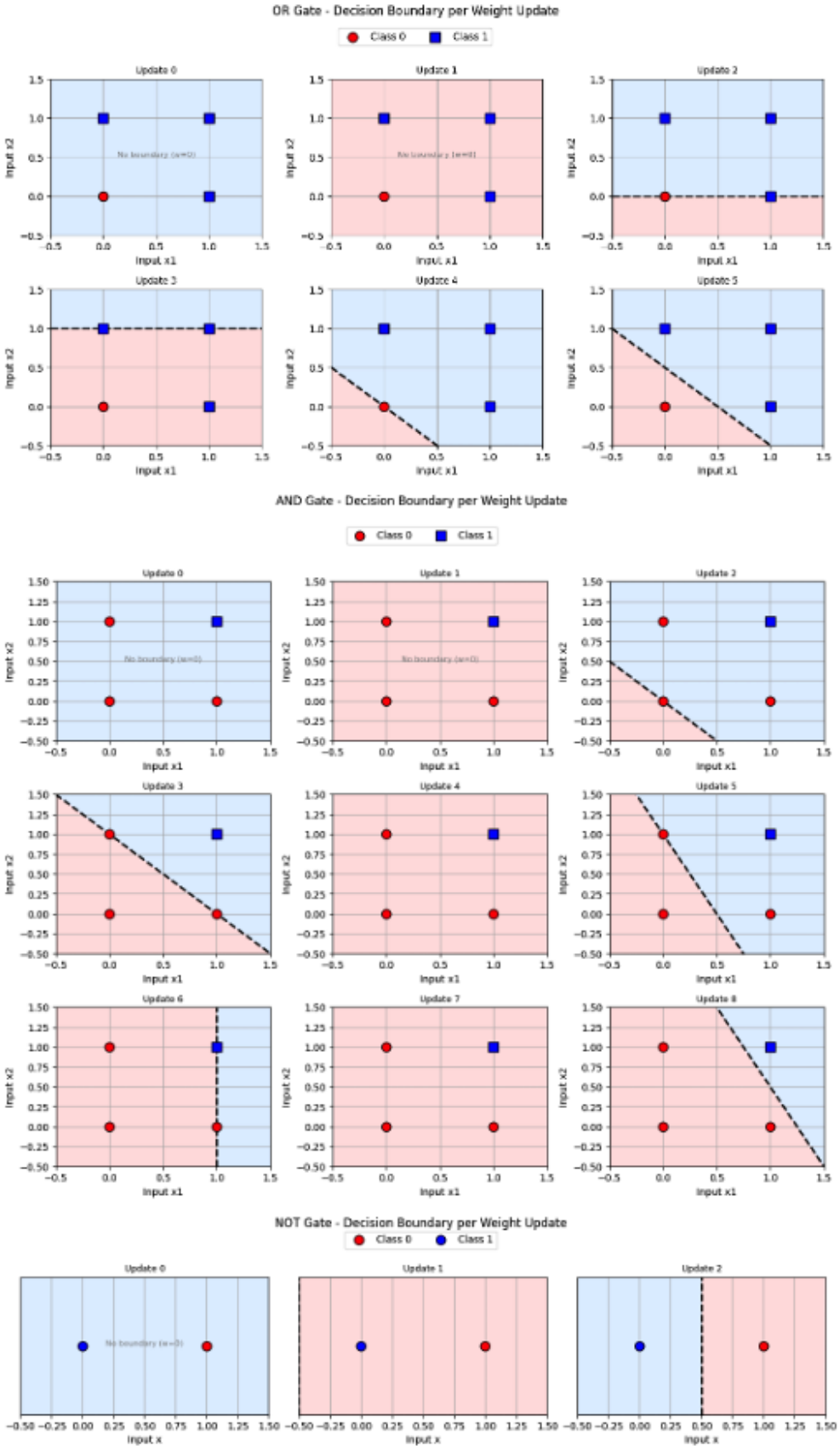


Figure 10: Decision Boundary After Every Weight Update for the OR, AND and NOT Gates

The combined figure displays all 15 decision-boundary snapshots recorded across the three gates (5 for OR, 8 for AND and 2 for NOT). For the OR and AND gates, the boundary line can be seen rotating and shifting after every update, gradually moving into a position that correctly separates the red (class 0) and blue (class 1) points; by the final snapshot of each gate, every point lies on the correct side of the line. For the NOT gate, the boundary is a single vertical threshold point which, after only two updates, correctly separates the single positive sample from the single negative sample on the number line. These visualizations confirm, sample by sample, exactly how the perceptron incrementally reshapes its decision boundary until the gate is learned perfectly.

10. Discussion

The implementation of the Single Layer Perceptron demonstrated that a simple linear classifier can effectively solve linearly separable binary classification problems, a conclusion reinforced across both the Banknote Authentication task and the OR, AND and NOT logic gates studied in this report. The Banknote Authentication dataset proved highly suitable for perceptron learning because the two classes become nearly separable after feature normalization, while the OR, AND and NOT gates confirmed the same principle in a simpler, fully controlled setting, since each is a classical example of a linearly separable Boolean function.

The training error decreased considerably over successive epochs on the banknote task, indicating that the perceptron successfully learned an appropriate decision boundary, and the gradual stabilization of both the weights and bias further confirmed convergence of the learning algorithm. The evaluation metrics indicate excellent classification performance, with the model achieving an Accuracy of 98.18%, Precision of 100.00%, Recall of 96.03% and an F1-Score of 97.98%, demonstrating that the implemented classifier generalized well on unseen testing samples. The learning rate experiment further showed that smaller learning rates (0.001 and 0.01) produced slightly better performance than a larger learning rate (0.1), since larger learning rates resulted in comparatively larger weight updates, producing a slight reduction in testing accuracy.

The logic gate experiments illustrate the same convergence behaviour at the level of individual weight updates. The OR gate converged fastest among the two-input gates, requiring only 5 updates, since its positive class occupies three of the four input corners, making it comparatively easy for a single line to isolate the sole negative example $(0, 0)$. The AND gate required more updates (8 in total) because its positive class is a single corner point $(1, 1)$, requiring the boundary to be pushed further away from the origin before all three negative points could be placed correctly on the negative side. The NOT gate, having only two possible input values, converged almost immediately in just 2 updates. In every case, the weights and bias changed monotonically towards a stable configuration and then stopped changing entirely once an epoch produced zero misclassifications, which is the defining signature of convergence, and these results align exactly with perceptron convergence theory, which guarantees that the algorithm will find a separating hyperplane in a finite number of steps whenever the underlying data is linearly separable.

Although the Single Layer Perceptron performs well on linearly separable datasets and logic gates alike, it cannot solve non-linearly separable problems such as XOR. Such problems require multilayer neural networks together with differentiable activation functions and backpropagation learning.

11. Conclusion

In this experiment, a Single Layer Perceptron was successfully implemented from scratch using Python in Google Colab and evaluated on two complementary tasks: binary classification on the Banknote Authentication dataset, and learning of the OR, AND and NOT logic gates. For the banknote classification task, the dataset was explored, preprocessed and normalized before training the classifier, and the perceptron learning algorithm successfully updated the weights and bias over multiple epochs, resulting in a well-trained binary classifier; the generated plots clearly illustrated the learning behaviour of the model through training error reduction, weight convergence, bias evaluation and learning-rate comparison, and the trained model achieved high classification performance with an Accuracy of 98.18%, demonstrating that the Single Layer Perceptron is an effective solution for linearly separable binary classification tasks.

For the logic gate task, the Perceptron Learning Algorithm was used to successfully learn the OR, AND and NOT gates, with the weights and bias logged after every individual update and the decision boundary visualized at every stage of learning, clearly illustrating how the perceptron incrementally rotates and shifts its boundary until all training examples are classified correctly; all three gates converged within a handful of epochs (3 to 4), confirming that the Single Layer Perceptron is a reliable and efficient solution for linearly separable binary classification problems of this kind.

Taken together, the experiment provided practical understanding of activation functions, perceptron learning, parameter updates and performance evaluation metrics, and demonstrated consistently, across both a real-world dataset and fundamental logic gates, that the Single Layer Perceptron is an effective and reliable solution wherever the underlying classification problem is linearly separable.

12. References

1. Frank Rosenblatt, *The Perceptron: A Probabilistic Model for Information Storage and Organization in the Brain*, Psychological Review, 1958.
2. Ian Goodfellow, Yoshua Bengio, Aaron Courville, *Deep Learning*, MIT Press, 2016.
3. Christopher M. Bishop, *Pattern Recognition and Machine Learning*, Springer, 2006.
4. Simon Haykin, *Neural Networks and Learning Machines*, Pearson Education, 2009.
5. UCI Machine Learning Repository, Banknote Authentication Dataset.
   https://archive.ics.uci.edu/dataset/267/banknote+authentication
6. Scikit-learn Documentation.
   https://scikit-learn.org